Tourist Safety Monitoring System — Final Technical Report

Repository: `D:\TouristSafetyMonitoring-src` (Android · Kotlin · Jetpack Compose · Room) Report date: 2026-10-16 (revised 2026-10-17 — see §7.7–§7.9, §8 and §12; revised 2026-10-18 — see §7.10, §7.11 and §7.12) Status: **buildable and tested on the JVM; on-device items are honestly marked NOT EXECUTED**

---

1. Executive summary

This pass fixed two real defects, added three new capabilities, and produced a reproducible data-analysis/machine-learning module plus full documentation — **without inventing a backend, a dataset, model metrics, or test results.** A later pass (2026-10-17) added real supervised models, the approach-risk briefing, and the crash fix for the reported "app closes by itself" (see §7.7–§7.9). A final pass (2026-10-18) added the one model the brief actually asked for and the project did not yet have: a **genuine supervised future-event predictor** (§7.10), trained on real recorded falls and run on-device. A further pass the same day added **self-service tourist registration** (§7.11): a tourist creates their own account from the phone, the service mints the Tourist ID, and the new account appears on the authority's dashboard with no officer action. The last pass (§7.12) fixed the reported dead submit buttons on a fresh install and split the app into **two roles** — a tourist journey and a separate, server-enforced **officer console** — with the backend address moved out of the tourist's path.

#	Deliverable	State
1	**Key-less map** — the Create Geo-Fence screen is usable with no Google Maps key (three independent key-free tile providers: OpenStreetMap and its mirrors, Esri World Imagery, OpenTopoMap; OSM is the default)	Implemented; projection + provider policy + offline tile cache + placeholder detection unit-tested; on-device tile rendering NOT EXECUTED
2	**Truthful background-service status** — Home now reports the *real* service state (RUNNING / PAUSED / OFF)	Implemented and driven by the real service lifecycle
3	**Offline outbox + transport** — events are queued durably and retried; no fake delivery	Implemented; a **real HTTP transport** delivers to the authority backend, or stays honest when no address is configured; 8 queue + 10 payload tests pass

#	Deliverable	State
4	**Simulated smartwatch** — SAFE/WARNING/SOS, location, latest alert, vibration, SOS button, honest delivery status	Implemented; 10 tests pass; vibration NOT EXECUTED (no device)
5	**ML / EDA pipeline** — real GeoLife data, unsupervised trajectory-anomaly scoring, model card	Implemented; real measured metrics
6	**Tests & docs**	**548** JVM tests pass (541 + 7 opt-in skips), **265** authority-backend tests, **41/41** integration checks, **179** ML tests pass, 10 technical documents
7	**Multiple areas + live follow** — tap-to-create sample zones, camera follows the newest fix	Implemented; geometry unit-tested (6 tests); UI wiring NOT EXECUTED
8	**Map usability bug-fix round** — visible blue "you are here" marker, closer default zoom, typed lat/long corners, crash log, bounded tile memory	Implemented; unit tests pass; marker/zoom/dialog NOT EXECUTED on a device
9	**Crash fix** — the reported "blue marker appears, then the app closes" (`CameraUpdateFactory is not initialized`)	Root cause identified, guards + `runCatching` added; compile/JVM-verified, device confirmation **NOT EXECUTED** (INST-16)
10	**Movement recognition + impact detection** — two supervised random forests trained on real UCI HAR data, exported to JSON, walked on-device in Kotlin (no TFLite)	Implemented; activity accuracy **0.8351** / macro-F1 0.8333; impact detector ROC-AUC 0.9999 vs shuffled control 0.5198; export verified in Python **and** Kotlin
11	**Historical risk analysis on approach** — a briefing shown within 200 m of a restricted/dangerous zone, with every risk factor itemised	Implemented; 26 unit tests; explicitly a published weighted checklist, **not** a learned probability
12	**Tracking UI** — movement/fall card and approach-briefing card, plus a distance-to-boundary pill over the map	Implemented; state maths unit-tested, rendering NOT EXECUTED
13	**Authority backend + operator dashboard** — one Flask app + SQLite: **self-service tourist registration**, tourist login (password or PIN), `POST /sync/events` with a per-event verdict (idempotent by `dedupeKey`), emergency contacts, and a dashboard with a **Registered Tourists** register and the SOS queue	Implemented; **265** backend rule tests + **41/41** real-subprocess acceptance; the production Kotlin client reached it **7/7** live; the device's own path is NOT EXECUTED (INST-20/21/24)

#	Deliverable	State
14	**Real transport wired into the app** — a transport selector + a dedicated session store + Authority and Emergency-contacts screens replace the hard-coded honest stub	Implemented; **51** client/transport/real-socket tests pass; screen rendering NOT EXECUTED on a device
15	**Routine position reporting (offline location sync)** — real-GPS fixes are queued as `POSITION` events behind an explicit interval/distance policy and delivered like any other event, so the authority's map tracks the tourist between alerts	Implemented; **12** new tests (reporter + policy) plus payload tests; simulated/out-of-range fixes are refused; device GPS NOT EXECUTED
16	**Tourist PIN + SOS acknowledgement + status indicators** — id + PIN sign-in (scrypt-hashed server-side), `PUT /me/pin`, the tourist seeing their SOS acknowledged, plus real GPS/authority status cards and responsive action/chip rows	Implemented; **18** backend PIN tests + app-side PIN/position payload tests; live-verified (the suite has since grown — see item 19); on-device rendering NOT EXECUTED
17	**Remaining EDA / history / model work** — real OpenStreetMap EDA (Overpass), real weather EDA (Open-Meteo), the UCI HAR figure set, historical trajectory + zone-crossing trends, and a supervised transport-mode model	Implemented as reproducible scripts; **86** ML tests pass
18	**Genuine future-event risk prediction** — a supervised random forest over **real recorded falls** (UniMiB SHAR), predicting a hard impact **within the next second**, calibrated, run **on-device**, shown to the tourist with its contributing factors and shared with the authority as a labelled prediction	Implemented; PR-AUC **0.764569**, trial sensitivity **0.929577**, shuffled-label control **0.477**; **94** new JVM tests + **93** new ML tests + **28** backend tests; **not a deployment-ready alarm** (5.09 false alarms/min) and **never triggers an SOS**; on-device rendering NOT EXECUTED
19	**Self-service tourist registration** — a tourist with no credential creates their own account from the phone (name, phone, a PIN they choose, optional emergency contacts); the **database** mints a unique `TR-<year>-<n>` Tourist ID; the success screen shows it with a **copy** button and sign-in instructions; signing in uses that id and PIN; the account appears under **Registered Tourists** on the dashboard with **registration**, **tracking** and the safety signals kept separate	Implemented; **45** backend tests + **24** new JVM tests + **1** new live E2E; the live run minted `TR-2026-00001` and read the dashboard row back; the registration **screen** is NOT EXECUTED (INST-24)

#	Deliverable	State
20	**Two-role sign-in, and the dead-button fix** — the app now opens on a **Welcome** screen offering *I'm a tourist* and *I'm an authority officer*; the tourist path leads to one combined **sign-in / create-account** screen (never gated on a backend address), the officer path to an **officer console** listing the register. The submit controls are gated on **form validity** with per-field messages and a "*still needed*" line, so nothing is ever silently greyed out. The tourist and officer sessions are stored under **separate keys** and are independent; the officer route is enforced **server-side** (a tourist token gets **403**). The address moves out of the tourist journey into *Settings → Backend connection (advanced)*, where **Test connection** really calls `/api/v1/health`, and a debug build can carry a first-launch default address that never overwrites a chosen one	Implemented; **+2** JVM classes and **+30** JVM cases (**49** classes / **545** cases at that point; now **50 / 548** after deliverable 21), **0** failures; the live client signed in as an officer, read the register and was refused the same route with a tourist token; the **Welcome**, **sign-in** and **officer console** screens are NOT EXECUTED (INST-24/25)
21	**The phone could not reach the authority: Android's cleartext block** — the app talks to a **plain-HTTP** LAN authority, and since Android 9 (API 28) the platform refuses cleartext HTTP for any app targeting 28+ unless the app opts in. With `targetSdk 35` and no profile, **every** call died in `Cleartext HTTP traffic to 192.168.1.102 not permitted` before a byte left the phone — which the sign-in screens could only report as "the authority is not answering". A `network_security_config.xml` now permits cleartext for the LAN authority, the manifest declares it, and a test fails the build if the pair ever drifts	Implemented; the fix is **inside the built APK** (`aapt2 dump xmltree` shows `android:networkSecurityConfig`, and the referenced `res/xml/network_security_config.xml` carries `cleartextTrafficPermitted`); **+3** JVM cases (**548** total); whether the banner clears is NOT EXECUTED (needs the phone)

Headline numbers (all measured here): Android JVM tests **548 / 541 pass, 7 opt-in skips**; authority backend **265** tests + **41/41** acceptance checks; ML tests **179 / 179 pass**; debug APK builds (**20.33 MB**, SHA-256 `F0786379…3BE5D239`); GeoLife **18,670 trajectories / 24,876,978 points** scanned with **0 malformed rows**; trajectory-anomaly injection ROC-AUC **0.8922** (shuffled control **0.5007**); UCI HAR activity accuracy **0.8351** on 2947 held-out windows from 9 unseen subjects; impact-detector ROC-AUC **0.9999** on **synthesised** impacts (shuffled control **0.5198**) — the caveat matters and is stated in full in `docs/model_card_movement.md`; future-risk model PR-AUC **0.764569** / ROC-AUC **0.936848** on 10,415 windows from 6 unseen subjects, **264 of 284** real held-out falls flagged (shuffled control **0.477**) — the false-alarm rate and the lab-data caveat are stated in full in `docs/future_risk_prediction.md`.

---

2. What was wrong (diagnosis)

2.1 The Create Geo-Fence map was blank with no Google key

`ui/components/SafetyMap.kt` gated the entire map on `mapsApiKeyConfigured` (`BuildConfig.MAPS_API_KEY` non-blank). With no key it rendered a "missing key" notice **instead of a map**, so taps never reached `onMapClick`, no vertices could be added, "Close ring" stayed disabled, and zones could not be saved. The key plumbing itself was correct; only the *no-key* path was broken.

2.2 Home contradicted itself: "Tracking: Active" but "Background service: Not running"

`HomeScreen.kt` printed `snapshot.backgroundTracking`, which is only the *intent* flag set by `TrackingController.start(backgroundTracking = …)`. It never reflected whether `LocationTrackingService` was actually alive, and it conflated three distinct states (tracking on / service on / service wanted-but-blocked).

2.3 No delivery path for alerts or SOS

There was no persistence for outbound events and no transport, so an alert or SOS generated while offline had nowhere to go. There was also **no backend server in this repository** — that gap is now closed: `backend/` is a working Flask + SQLite service with an operator dashboard, and the app has a real transport wired to it (deliverable 13 and 14 below, and `docs/progress_log.md` §12).

---

3. Deliverable 1 — Key-less map (OpenStreetMap fallback)

Design. When a Google Maps key is present, the app uses Google Maps exactly as before. When it is absent, `SafetyMap.kt` renders `OfflineOsmMap`, a self-contained Compose canvas that:

- fetches standard OpenStreetMap raster tiles;
- projects lat/lon to screen with a pure, unit-tested `MercatorProjection` (integer zoom levels so tiles and geometry align exactly);
- supports pan/zoom/tap gestures and the same tap-to-add-vertex flow;
- **degrades to a graticule grid** if tiles cannot be fetched (offline), so the screen is **never blank** and zones can still be drawn and saved.

Honesty. The projection maths is proven by `MercatorProjectionTest` (11 tests: known slippy-map tile indices, screen↔world round-trips, ground resolution at the equator, latitude clamping,

panning). **Whether tiles visually load is NOT EXECUTED** — this environment has no device/emulator and no way to observe rendering. Do not treat the map as device-verified.

Files: `ui/map/MercatorProjection.kt` (new), `ui/components/OfflineOsmMap.kt` (new), `ui/components/SafetyMap.kt` (modified).

3.1 Usability round after real-device feedback (2026-10-17)

Four defects reported from a phone, each traced to a concrete cause:

Report	Cause found	Fix
"the blue mark is not visible"	`UserMarkerAccent` was `0xFF0B6E4F` — the theme's **dark green**, and `SafeStroke` is `0xFF1B7F4C`, so the "you are here" dot matched a **SAFE zone's fill**	Marker colour changed to the conventional blue `0xFF1A73E8` with a dark outline and white collar; a **hollow grey** ring marks a last-known-only position; a status line states whether a live fix exists
"in seconds app closed automatically"	Not reproducible here (no device, empty `adb devices`). Two real memory paths removed: a coroutine per tile **per camera change**, and an in-memory tile map that was **never pruned**	140 ms de-bounce, batches of 4, off-screen tiles dropped, `LruCache` sized in bytes, providers skipped once dead (`TilePolicy.providersToTry`); **plus** a crash log + on-launch dialog so the real cause can be identified (`diagnostics/CrashLog.kt`)
"the map view shld be close not that wide"	Default zoom was 15 — about a town on one screen, so a tapped-out boundary was far larger than intended	Default zoom 15 → **17**; editor map 280 → 320 dp; already-saved zones are drawn behind the draft
"need 2 time in manual entry and latitute and logitude entry"	A refused tap was **silent**, and there was **no** coordinate entry at all	Refusals now explain themselves; a Latitude/Longitude row with **Add point** and **Fill from my position** was added (the fill button is what makes southern/western signs workable, since a phone's decimal keypad has no minus)

Honesty. All four are **logic-verified, rendering NOT EXECUTED**: the crash diagnosis is explicitly a plausible-cause fix plus instrumentation, not a confirmed root cause, and the marker/zoom/dialog can only be judged on a phone.

Follow-up (later on 2026-10-17) — the crash was then root-caused. The crash log added above did its job: the in-app dialog captured

```
java.lang.NullPointerException: CameraUpdateFactory is not initialized
  at com.google.android.gms.maps.CameraUpdateFactory.newLatLng
  at com.touristsafety.app.ui.components.SafetyMapKt$FollowLatestFix$1$1.invokeSuspend(Saf
```

so "in seconds app closed automatically" was **not** a memory problem: with no Google Maps key the Play Services map SDK is never initialised, but the live-follow effect still called `CameraUpdateFactory.newLatLng` on every fix — which is exactly why it crashed the moment the blue marker first appeared. Fixed with a `googleCameraReady` guard plus `runCatching` in `followLatestFix`, `moveTo` and `fitPoints` (and an early return on degenerate bounds). Compile- and JVM-verified; confirming the *reported* crash is gone needs a device, so it is recorded as **INST-16 NOT EXECUTED**.

---

4. Deliverable 2 — Truthful background-service status

Design. `LocationTrackingService` sets a real flag only after `startForeground()` succeeds and clears it in `onDestroy()`. `TrackingController` exposes that flag. `HomeViewModel` maps the three real-world states to an explicit `BackgroundServiceStatus`:

- **RUNNING** — the foreground service is genuinely alive;
- **PAUSED** — background tracking was requested but the OS never granted background location;
- **OFF** — not requested.

Home now shows the truth, resolving the self-contradiction.

Files: `tracking/TrackingController.kt`, `tracking/LocationTrackingService.kt`, `ui/viewmodel/HomeViewModel.kt`, `ui/screens/HomeScreen.kt` (all modified).

---

5. Deliverable 3 — Offline outbox + pluggable transport

Design. Every alert/SOS is written to a durable local **outbox** first, then handed to a `SyncTransport`.

- **Durable:** table `outbox_events`, added by an additive migration (`MIGRATION_1_2`); existing zones and history are **not** erased.
- **Exactly-once semantics on the client:** idempotent enqueue on a stable dedupe key; delivered rows leave the pending queue; permanent rejections are marked failed; transient failures stay pending and increment the attempt count.

- **Honest transport, now real:** `SyncTransport` is an interface. The shipped `SelectingSyncTransport` delegates to the real HTTP authority transport when a base URL is configured and to `NoBackendTransport` (which returns `Retry`) otherwise. **The app never claims a message was delivered when it wasn't** — with no address configured the status reads "Queued (offline)"; with an address it reports the server's actual verdict.

Schema changes are versioned (`AppDatabase.VERSION = 3`): `MIGRATION_1_2` adds the outbox and `MIGRATION_2_3` adds the emergency-contact table, both additive, with no `fallbackToDestructiveMigration()`; each migration's SQL was verified against the generated `N.json` and is executed on the JVM by `RoomPersistenceTest`.

Files: `domain/model/OutboxEvent.kt`, `data/database/entity/OutboxEventEntity.kt`, `data/database/dao/OutboxEventDao.kt`, `data/mapper/OutboxEventMappers.kt`, `data/repository/OutboxRepository.kt`, `outbox/OutboxPayloads.kt`, `outbox/SyncTransport.kt`, `outbox/OutboxSyncer.kt`, `data/database/DatabaseMigrations.kt` (all new); `data/database/AppDatabase.kt`, `AppContainer.kt` (modified). The authority client itself is described under deliverable 14 and in `docs/progress_log.md` §12.

---

6. Deliverable 4 — Simulated smartwatch

Design. A phone-screen "watch" that consumes the **same real** `TrackingSnapshot` (no second data source):

- face: **SAFE / WARNING / SOS**;
- shows the current location and the latest alert;
- **haptic on entering a restricted/dangerous zone** — exactly once, behind a `WatchHaptics` interface (`AndroidWatchHaptics` in production), so it does not buzz on every fix;
- a distinct, longer **SOS** vibration pattern;
- a **SIMULATED SOS** button that writes an SOS into the outbox (origin `SIMULATED_WATCH`);
- honest delivery status — "Queued" with the reason until an address is configured and a tourist is signed in, then the **real** verdict from the authority;
- stamped unmistakably **"SIMULATED WATCH — NOT PHYSICAL HARDWARE"** so it can never be mistaken for a real emergency device.

Files: `watch/WatchUiState.kt`, `watch/WatchHaptics.kt`, `watch/AndroidWatchHaptics.kt`, `watch/WatchController.kt`, `ui/viewmodel/WatchViewModel.kt`, `ui/screens/WatchScreen.kt` (all new); `navigation/AppNavigation.kt` (WATCH route + top-bar action), `AndroidManifest.xml` (`VIBRATE`) — modified.

---

7. Deliverable 5 — ML / exploratory-data-analysis pipeline (`ml/`)

Self-contained under `ml/`; **supplements — never replaces** the deterministic geo-fencing in `app/` and does not touch `app/`.

7.1 Dataset (real, verified)

Microsoft Research **GeoLife GPS Trajectories 1.3** — the official archive was **downloaded and unzipped here** (313,164,406 bytes, 298.7 MB). Measured scale: **182 user directories, 18,670 `.plt` files, 24,876,978 data lines, 0 malformed rows**. The guide claims 17,621 trajectories while the archive contains 18,670 — **both are reported**. No open-source licence is bundled; use is research-only with the required citation.

7.2 Verified schema and a real data quirk

`.plt` = 6 header lines + CRLF + 7-token data lines; the two independent time encodings agree to within 4.8×10^{-6} s. **Critical quirk:** `.plt` file names collide across users (847 ids shared), so every grouping uses `traj_uid = user_id | trajectory_id` — otherwise different people's trajectories would silently merge (a leakage bug).

7.3 Cleaning & EDA

Cleaning removes unparsable timestamps, out-of-range/null-island coordinates, duplicate `(trajectory, timestamp)` rows (698,900 removed), sorts and segments on gaps > 120 s, and flags GPS jumps (> 60 m/s). EDA produced **12 figures + 11 tables + a findings document** from real values (e.g. median sampling interval 2.00 s; median speed 3.23 m/s; busiest hour 09:00). Full detail in `docs/dataset_and_eda.md`.

7.4 Model

Unsupervised `sklearn.ensemble.IsolationForest` (200 trees, seed 42 — reproducible) over 14 standardised movement features. Output = **movement anomaly score** (higher = more unusual), **not a probability** and **not danger**. Training used a 1,200-trajectory fixed-seed sample, split **by `traj_uid`** (900 train / 300 test, 0 overlap).

7.5 Evaluation — honest by construction

GeoLife has **no anomaly/danger labels**, so accuracy/F1 against real anomalies **cannot be computed** and is **not reported**. Validation uses **real held-out data with synthetic labels** (injected GPS teleports):

Metric	Value
ROC-AUC	**0.8922**
Average precision	**0.6763**
Precision@k / Recall@k (k = 17,462)	**0.6165** / **0.6165**
Label-shuffle control ROC-AUC	**0.5007**

A high score means teleport artefacts are separated well — **not** that real danger is detected. Full detail and limitations in `docs/model_card.md`.

7.6 The SYNTHETIC fallback

A tiny, clearly-labelled synthetic fixture exists only so the pipeline and tests run on a machine without the 299 MB dataset. Every stage prints a loud SYNTHETIC banner and writes a distinct `SYNTHETIC/` directory. **Synthetic numbers are plumbing tests, never results.**

7.7 Movement recognition and impact detection — UCI HAR (supervised, shipped on the device)

The GeoLife work above is GPS-only and unsupervised: it can say "this trajectory is unusual", which is not the same as **what the person is doing** or **whether they just fell**. That needed a second, supervised pipeline on a sensor dataset — the **UCI HAR** dataset listed in `DATASETS.pdf`.

Two scikit-learn models are trained, exported to JSON, shipped in `assets/`, and walked on the phone by hand-written Kotlin (`motion/DecisionForest.kt`). **No TensorFlow / TFLite / PyTorch, and no network at inference time** — the two models together are under 330 KB, which is why a hand-written walker was chosen over a runtime.

Features. The phone computes **24 time-domain features** (`MotionFeatures.kt`) that mirror the Python extractor exactly, in the same order — validated on both sides of the wire. Accelerometer samples are converted to *g* by dividing by 9.80665 m/s² so the units match the dataset. The 561 UCI features (FFT energy bands, entropy) are not computable in real time on a phone.

Model A — activity recognition (real human labels). Random forest, 32 trees, depth 8.

Model	Features	Accuracy	Macro-F1
Shipped on-device model	**24**	**0.8351**	**0.8333**
561-feature reference	561	0.9287	0.9271
Single decision tree (baseline)	24	0.7967	0.7943

Evaluated once on **2947 held-out windows from 9 subjects never seen in training** — the dataset's own subject-disjoint split, verified as `subject_overlap: []`. Adding the three per-axis means (the gravity direction, i.e. how the phone is held) is what carried the on-device model to 0.8351; the three walking variants are the confusion, and the app only ever asks the model the coarser question "is this person moving?" (`ActivityKind.isAmbulating`), which is exactly the distinction that is reliable.

Model B — impact / fall-signature detector (synthesised positives). Random forest, 40 trees, binary.

> **UCI HAR contains no falls.** There is no fall class, no impact annotation and no subject who fell. So the positive class was **manufactured**: a fall-like transient (short free-fall dip, impact spike of several g, then stillness) was injected into real recorded windows from the held-out subjects; the negative class is those same recordings untouched.

Metric	Value
ROC-AUC	**0.9999**
Average precision	**0.9999**
F1 @ 0.5	**0.9962** (precision 0.9957, recall 0.9968)
Shuffled-label control ROC-AUC	**0.5198**

The last row is what gives the others meaning: the identical pipeline on randomly permuted labels collapses to chance, so the near-perfect score is the detector finding the injected signature in real held-out recordings — not leakage or a label-encoding feature. **It is still not a fall-detection claim**: an injected impact is a large stereotyped transient, and a real fall can be gentle or look like sitting down quickly. The named upgrade path is **SisFall / UniMiB SHAR / MobiFall** (real falls), and it has deliberately not been claimed as done. The detector's score is also never sufficient on its own — see §7.9.

An exporter bug was caught before it shipped: scikit-learn's `tree_.value` holds **class fractions, not counts**, so `np.rint(tree.value)` collapsed every leaf to a one-hot vector. The model would have loaded, run, and answered — just wrongly. Fixed to `np.rint(tree.value * tree.weighted_n_node_samples)`, with a pytest regression guard, and the exported JSON is verified against scikit-learn (max probability error **4.97e-07**, **0** argmax mismatches) both in Python and, independently, in Kotlin against the shipped asset.

Full method, per-class tables, embedded confusion matrices, limitations and the reproduction commands: **`docs/model_card_movement.md`** .

7.8 Historical risk analysis on approach — deliberately **not** a model

When the tourist comes within 200 m of a restricted or dangerous boundary, the Tracking screen now shows an **approach briefing**: the zone, the distance, the tourist's *own* recorded crossings of that zone, and a 0–100 risk score with **every factor itemised and explained**.

The score is a **published weighted checklist** (`ApproachRiskModel`), not a learned danger probability: zone severity (DANGEROUS 40 / RESTRICTED 28 / SAFE 5), 3 points per crossing in the last 7 days capped at 18 (4 points for never-visited ground), proximity up to 16, night 12 / dusk 5, ambulating 4, and the live impact score 12 above threshold (6 neutral when no model is available). Bands are LOW < 30 ≤ MODERATE < 60 ≤ HIGH.

It is deliberately not dressed up as a probability, because the app has **no labelled outcome data** and there is nothing to fit. The weights live in one place and are unit-tested (26 tests across `ApproachRiskTest`, `ZoneHistoryTest`, `ApproachBriefingTest`), including the invariants that the score never exceeds 100 and that the listed factors account for every point of it. The card says this to the user's face rather than only in the docs.

7.9 How a fall is actually decided on the phone

The model's score is **one of three gates**, and all three must agree before anything is reported:

Gate	Threshold	Why
Impact magnitude	≥ **2.0 g**	a hard fall lands roughly 2–5 g; below that it is a stumble or a dropped phone
Model score	≥ **0.55**	the window looks like the trained impact signature
Stillness after the impact	std ≤ **0.12 g**	measured from the *last* sample at or above the impact threshold, so the impact plateau itself is not mistaken for stillness
Cooldown	15 s	overlapping windows mean one fall would otherwise raise five alerts
Tail fraction	0.25	if the peak lands in the final quarter there is not enough signal to judge stillness, so that gate is **skipped**, not failed

Consequences visible in the UI: the face shows the probability **and** its threshold, the peak g, the stillness value and its threshold, and it says **"suspected"**. A high model score with no physical impact raises nothing, and **no SOS is ever sent automatically**.

7.10 A genuine supervised *future* risk prediction — on-device, and honest about its basis

Everything above is either a configured rule (§7.8's approach checklist) or a detector that reacts to an impact that **has already happened** (§7.9). This round adds the third, genuinely different thing the brief asked for: a **trained model that predicts an event before it happens**, and it deliberately does *not* pretend the project's four named datasets can support it.

The dataset problem, stated plainly. GeoLife, UCI HAR, OpenStreetMap and OpenWeatherMap/Open-Meteo contain **no outcome labels at all** — no injuries, no incidents, no falls. A supervised tourist-incident model cannot be trained from them, and inventing labels would be fabrication. The honest options were to find a dataset with real outcome labels or to ship nothing; this round did the former and documents the swap.

The dataset actually used. UniMiB SHAR (Micucci, Mobilio, Napoletano, 2017) — 30 subjects, 9 activities of daily living and 8 real fall types, 50 Hz phone accelerometer, with **real recorded falls**. 1,980 trials were read; **1,979 used**. One recording (subject 10, `Jumping`, trial 1) contains 9,369/9,624 non-finite values and was **dropped, counted and reported**, not repaired. The dataset has no OSI licence; the provenance file says so rather than inventing one. Full provenance is in `docs/future_risk_prediction.md` §2.

The target and the horizon. `IMMINENT_FALL` = *a hard fall impact lands within the next second*. Impact instant = `argmax(magnitude)` of the trial, accepted at ≥ 2.0 g (the app's own `FallPolicy.MIN_IMPACT_G`); 6 fall trials never reach 2 g and are excluded and counted. Horizon and feature window are both **50 samples = 1.0 s**, chosen because fall trials run 151–446 samples with the impact around index ~120, leaving ≥ 95 pre-impact samples — so **1.0 s is the longest fully supported horizon**, not a round number. **A 30-minute horizon is explicitly not attempted**, because no accelerometer can see 30 minutes ahead.

No leakage, checked not asserted. A window ending at `t` uses samples `[t-49, t]`; the label asks about `(t, t+50]`. The two intervals are disjoint, `train.check_no_future_information()` re-derives that from the raw recordings on every run, and a **shuffled-label control** (same features, same pipeline, permuted labels) scores **ROC-AUC 0.477** — chance, as it must be if nothing leaks.

Method. Subject-disjoint 60/20/20 split, seed 42, 30 subjects → 18/6/6 (**disjoint: true**); 51,128 windows (6,458 positive / 44,670 negative); `class_weight="balanced_subsample"`. **No scaler is fitted at all** — every feature is scale-free within its own window — so there is no fitted preprocessing step that could carry a global mean from train to test. Validation subjects were used only for the Platt calibration, the two band thresholds and the hyper-parameter search; the test subjects were scored once.

Measured, on held-out subjects (10,415 windows, 6 unseen subjects, 1,339 positive):

Model	Precision	Recall	F1	ROC-AUC	PR-AUC
Majority class	0.0000	0.0000	0.0000	0.5000	0.128565

Model	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic regression	0.4760	0.8432	0.6085	0.904389	0.659633
Hist gradient boosting	**0.7916**	0.6721	**0.7270**	**0.945913**	**0.815000**
Random forest, calibrated, shipped	**0.6707**	**0.7498**	**0.7080**	**0.936848**	**0.764569**

Plus the numbers that matter more than a window metric: **trial sensitivity 0.929577** (264 of 284 real held-out falls flagged at least once), and an **honest false-alarm rate of 5.09 per minute** of ordinary activity at the app's cadence (16.30/min at the raw window stride), concentrated in *jumping*, *sitting down* and *lying down* — the three activities where a body really does drop under gravity.

Model selection under a real deployment constraint. 65 configurations were searched on validation only. The quality winner (300 trees, unbounded depth, val PR-AUC **0.875909**) exports to **20,059.7 KB** and cannot ship to a phone; the chosen configuration (120 trees, depth 11, leaf 4, val PR-AUC **0.845112**) exports to **1,968,314 bytes** and ships at **1,974,297 bytes** with its metadata, against a **2,000,000-byte** budget that only **1 of 65** configurations satisfies. The **−0.031 PR-AUC cost is stated in the model card**, along with the fact that `hist_gradient_boosting` (0.8150) beats the shipped model and is *not* shippable on-device. The selection rule lives in `tuning.json` and is re-derived by a test.

Calibration. Platt scaling on `logit(p_forest)`, fitted on the **validation split only** (a = 1.249403, b = −1.079049), improves the Brier score from **0.062600** → **0.056675**. The reliability table is published *including* the bins that are over-confident (0.2–0.4), because hiding them would be the dishonest choice. Bands: **HIGH ≥ 0.314046** (validation max-F1) and **MODERATE ≥ 0.069201** (largest validation threshold keeping recall ≥ 0.95), both fitted on validation and shipped inside the artefact.

Where inference runs: on the device. The model reuses the app's **existing** `DecisionForest` JSON walker (the only change is an optional `expectedFeatures` parameter so a 22-feature model loads without weakening the existing 24-feature check) — **no new ML framework, runtime or format was added**. It is fed by the **same** acceleration subscription the movement watch already owns, taking the last 50 samples of each window, so there is no second listener and no extra battery cost. That makes the tourist-facing prediction work **fully offline**, and means no accelerometer data leaves the phone.

Three things kept visibly apart. The app now shows, as separate cards: the **configured geofence** status, the **approach-risk checklist** (§7.8, a published heuristic with no probability), and the **future risk prediction** (this model — the only one of the three that is trained, and the only one that shows a probability). The card's own text names the other two and denies being them; it carries the non-causal caveat for its factor list and states that it **never sends an SOS by itself**. Both sentences are asserted by tests, and `backend/tests/test_risk.py` asserts that a HIGH prediction leaves `latestSos` empty and `state != "SOS"` — **risk can never raise an SOS**.

What reaches the authority. A throttled **summary** (never raw sensor data) travels through the existing offline outbox as a new `OutboxKind.RISK` event with `origin = ON_DEVICE_MODEL` (`isSimulated = false`): first elevated band, any escalation, a 120 s refresh, and one "cleared" record when the band drops back to LOW. `UNAVAILABLE` and stale bands are never sent; the backend **rejects** `UNAVAILABLE` outright. Predictions are stored in their own `risk_assessments` table and shown in their own dashboard panel and tiles (`GET /api/v1/authority/risk`, per-tourist history, `latestRisk`) — never mixed into the SOS queue or the incident feed. This is what the earlier round's database gap was about: the dashboard previously had no table that could hold a prediction, so it neither received nor displayed one.

The honest bottom line. This is a real model, with real held-out numbers, real leakage controls and a real on-device path — and it is **not a deployment-ready fall alarm**. 5.09 false alarms per minute of ordinary movement, mid-range probabilities that are over-confident, lab-recorded data from 30 adults, and an untestable phone-drop-vs-person-fall failure mode are all in the model card. It is reported to the tourist as an informational state and to the authority as a labelled prediction, and it changes no operational state.

Not verified: the card's rendering on a real screen and the whole sensor→forest→card chain on real hardware. No emulator, AVD or device exists in this environment (`adb devices` is empty); the wording is instead pinned by `FutureRiskCardModelTest` (18 tests) and the inference by `FutureRiskReferenceTest`, which replays the *shipped* asset against scikit-learn's own probabilities for 16 real held-out windows.

Full model card, dataset provenance, measured metrics, calibration table, false-alarm breakdown and the complete limitations list: **`docs/future_risk_prediction.md`**. Everything is reproducible with `python ml/future_risk/train.py` and `powershell -File ml\run_all.ps1 -FutureRisk`.

7.11 Self-service tourist registration — the tourist creates their own account

Every tourist account in this project until now was created **by an officer** at the dashboard, and the tourist signed in with an id and a one-time password. That is the wrong shape for a tourist who simply installs the app: with nobody at a desk, there was no way to start. This round adds the missing journey — **register from the phone** — and makes the result visible to the authority without any officer action.

The ID is minted by the database, not the client. A client-side counter is the classic bug here: two phones registering in the same second both compute "00124". Instead a new table, `tourist_id_sequence (id INTEGER PRIMARY KEY AUTOINCREMENT, issued_at INTEGER NOT NULL)`, is the ledger: each registration consumes one row and reads the number SQLite issued. The text id is `TR-<current UTC year>-<5-digit number>` — e.g. `TR-2026-00124` — and because `tourists.id` is a **primary key**, even a text collision cannot produce a duplicate: the code steps over to the next

number. The counter is never reset, across years or restarts. `test_registration.py` registers twelve tourists in a row (twelve distinct ids) and forces a collision to prove the step-over.

The PIN is the credential, and nothing secret is returned. Self-registration keeps the existing `NOT NULL` `password_salt` / `password_hash` schema (no migration risk to existing rows) by writing a **random, immediately discarded** secret into those columns; the account's real credential is the PIN, through the project's existing scrypt `hash_password`. The response contains **only** the id and the account's public fields — no token, no password, no PIN, no salt, no digest — and **registering does not sign anybody in**. The tourist signs in as a separate step with the id and the PIN they chose.

One public write, and the escalation it refuses. `POST /api/v1/auth/register` creates only `tourists` rows, reads **no privilege field** (a body carrying `"role": "authority"` is `400`, and there is no field that could grant one — the path does not exist rather than being blocked), issues no session, and treats a phone number that is already registered as `409`. Validation is pure and lives in one place, `backend/tsm/registration.py`, reusing `hash_password` and the same contacts validator the outbox uses, so a contact accepted here is a contact the sync path would accept.

The app. The Authority screen's sign-in card gained two deliberate modes — *Sign in* and *Register as tourist*. The form collects a name, phone number, a chosen PIN and its confirmation, and any number of emergency contacts (added one at a time, removable). It is validated on the phone first (a mistyped confirmation is answered instantly, with no round-trip) and the server's `409` is shown as "*already registered*" rather than a generic failure. On success the card becomes a **registration-success screen** showing the id in a monospace readout with a **Copy Tourist ID** button, plus a plain-language instruction on how to sign in next (`RegistrationForm.kt`, `AuthorityContract.kt` / `AuthorityApi.kt` / `AuthorityClient.kt`, `AuthorityViewModel.kt`, `ui/screens/AuthorityScreen.kt`).

The dashboard. `GET /api/v1/authority/tourists` was enriched **additively** and the panel became **"Registered Tourists"**. Each row separates three different facts that used to be conflated:

- **registration** — is this a real account? (`registrationStatus`, `registeredAtMillis`, `registrationSource`: `SELF` for an app registration, the officer's id for a desk account, `null` older);
- **tracking** — have we heard from the phone? (`trackingStatus`, `lastSeenMillis`, `hasPosition`, `latestPosition`);
- **safety** — `openSosCount`, `latestRiskBand`, `latestRiskAtMillis` (unchanged in meaning).

A tourist who has registered but never reported reads **"Location not yet available"**; a position is never invented. New registrations appear at the top of the newest-first list on the next refresh, with no officer action.

A latent bug found on the way. While extending the register, the `RISK` ingest was found to insert its `risk_assessments` row **outside** the `with db().write()` block that inserted its `events` row, so a

prediction and its audit entry could commit separately. It now runs in the same transaction. The prediction itself is unchanged.

What was run. Backend **265** tests (+45), acceptance **41/41** (+8), Android JVM **515** tests (+31 across five classes — the suite has since grown to **548**; see §7.12–7.13 for the next rounds), and the opt-in live test against a real `run.py` **6/6** at that point (now **7/7**) — the new case registered `TR-2026-00001` with no credential, signed in with that id and the chosen PIN, read the row back from the officer's authenticated API (`registrationStatus=REGISTERED`, `trackingStatus=AWAITING_FIRST_REPORT`, `hasPosition=false`, `latestPosition=null`), then delivered a real-GPS report and watched the same row become `trackingStatus=OK`, `hasPosition=true`, `position={18.5204, 73.8567, simulated:false}`.

Not verified: the registration **screen** itself (the mode toggle, the form, the success card and its copy button) was never rendered, and the dashboard panel was never opened in a browser — there is still no emulator, device or browser in this environment (INST-24). The logic is JVM-tested, the wire format is real-socket-tested, and the whole journey runs through the **production** client against a real server on the JVM.

7.12 Two-role sign-in, and the button that was dead on a fresh install

The reported symptom. On a phone with a fresh install, the app opened on the *Authority* screen with **NOT SIGNED IN** and both **Sign in with PIN** and **Create my account** greyed out. Nothing on the screen explained why, and no Tourist ID was ever generated.

The root cause, exactly. Those two controls — and *Save address* — were enabled with

```
enabled = !busy && address.isNotBlank()
```

and `address` was seeded from the stored session's `baseUrl`, which is **empty on a fresh install**. So the first thing a new user saw was three dead buttons whose only requirement was an address that the app never asked for and never explained. It was not a registration bug and not a credential bug: the *submission* controls were gated on a **field the tourist was never meant to fill in**. Two further consequences fell out of the same code: the sign-in and registration flows lived on the same screen as the operator's backend configuration, and a tourist could reach the authority configuration at all.

The fix.

1. **The gate moved to form validity.** `RegistrationForm.fieldErrors(...)` now reports a problem *per field* and `RegistrationForm.outstanding(...)` lists only what is genuinely missing; the button is live whenever the form is not in flight, and pressing it with a bad form shows the named problems instead of doing nothing. **No submit control anywhere depends on a backend address.**

2. **The address left the tourist's path.** It lives in *Settings → Backend connection (advanced)*, where **Test connection** performs a real `GET /api/v1/health` — *Connected* is reported only when something actually answered, and an address with nothing behind it reports *Offline*.
3. **A build-time default, seeded once.** `-PAUTHORITY_URL` (or `TSM_AUTHORITY_URL`, or `AUTHORITY_URL=` in `local.properties`) becomes `BuildConfig.DEFAULT_AUTHORITY_URL`; `AppContainer.start()` writes it with `seedBaseUrlIfUnset`, which is a no-op when the user has already chosen an address. The APK attached to this pass carries `http://192.168.1.102:8080`, so a first launch on this machine's Wi-Fi is already pointed at the server.
4. **The two roles became two sessions.** The app opens on a **Welcome** screen with *I'm a tourist* and *I'm an authority officer*; a returning tourist with a valid session is offered *Continue* above the choices. Tourist and officer sessions are stored under **separate keys** in the same DataStore, so an officer sign-in does not make the phone a tracked tourist, and either sign-out leaves the other alone.
5. **The role boundary is the server's.** The officer console reads `GET /api/v1/authority/tourists`, which already required `require_auth("authority")`. Hiding the screen in the app proves nothing; the live test presents a **tourist** token to that route and gets **403** (§9).

What was run for this round. Android JVM **545 tests, 0 failures, 0 errors, 7 skipped** across **49 classes** (the suite has since grown to **548 / 50 classes** — see §7.13) (up from 515/47: **+30 cases** — `OfficerViewModelTest` 10 new, `AuthoritySessionStoreTest` 8 new, `RegistrationFormTest` +5, `AuthorityApiTest` +4, `AuthorityHttpServerTest` +2, `AuthorityLiveBackendTest` +1). The backend (**265**), acceptance (**41/41**) and ML (**179**) suites were re-run unchanged and stayed green. The live suite was run against a real `run.py` with `-Dtsm.e2e.baseUrl` and reported **7/7, 0 skipped**.

A genuine finding on the way. The session store's constructor took a `Context` only to derive a default `DataStore`, which made it untestable off-device. It now takes the `DataStore` directly, with `createAuthoritySessionStore(context)` as the production factory — which is what makes "the session survives a restart" and "the officer session is separate" *assertable* rather than asserted-by-inspection. The new `AuthoritySessionStoreTest` drives a real `PreferenceDataStore` over a throw-away file on the plain JVM, with no Robolectric.

Not verified: the Welcome screen, the tourist sign-in/create-account screen and the officer console were never rendered — there is still no emulator or device here (INST-24/25). Their logic is JVM-tested, the register route is real-socket-tested both ways, and both client methods run against a real server.

7.13 The phone could never reach the authority — and it was Android, not the server

The report. Two screenshots taken on a real phone. The tourist sign-in screen showed the red banner "*The authority at http://192.168.1.102:8080 is not answering. Check that the server is running and that this phone is on the same Wi-Fi network.*" and the connection card read **Status: Offline**. The

officer screen showed the same address with the raw sentence "*Cannot reach http://192.168.1.102:8080: **Cleartext HTTP traffic to 192.168.1.102 not permitted***".

The root cause, exactly. The second sentence is not written by this app — it is Android's. The app talks to its authority over **plain HTTP** (`http://192.168.1.102:8080`); the client wraps any `IOException` as `Unreachable` and prints the platform's message verbatim. **Android 9 (API 28) and later refuse cleartext HTTP by default for any app whose `targetSdk` is 28+**, this app targets **35**, and the manifest declared neither `android:usesCleartextTraffic` nor `android:networkSecurityConfig`. The request was therefore killed by the **network-security policy before it reached the network**: the server was never asked, never answered, and never logged a request from the phone — which is precisely what its access log shows. The tourist screen's softer "is not answering" is the same failure, summarised. This was true from the first build; restarting `run.py` could never have changed it.

The fix.

1. **`app/src/main/res/xml/network_security_config.xml`** (new) — a `<base-config cleartextTrafficPermitted="true">`, with the reasoning written into the file so a later "hardening" pass cannot silently re-break the app, plus what to do if the authority ever gains TLS.
2. **`AndroidManifest.xml`** — the `<application>` now declares `android:networkSecurityConfig="@xml/network_security_config"`.
3. **`NetworkSecurityConfigTest`** (new, 3 cases) — reads the shipped manifest and profile and fails the build if the manifest stops pointing at a cleartext-permitting profile, or re-blocks it with `usesCleartextTraffic="false"`.

What was run for this round. Android JVM **548 tests, 0 failures, 0 errors, 7 skipped** across **50 classes** (**+3**, all `NetworkSecurityConfigTest`). The backend (**265**), acceptance (**41/41**) and ML (**179**) suites were re-run unchanged and stayed green. The built artefact was verified directly: `aapt2 dump xmltree` shows `android:networkSecurityConfig(0x01010527)=@0x7f0c0002` on `<application>`, and `aapt2 dump resources` resolves `0x7f0c0002` to `res/xml/network_security_config.xml`, whose bytes contain `cleartextTrafficPermitted`. The opt-in live suite was **not re-run** this round (the backend was stopped), so its seven cases skip in this run; their last measured result is §7.12's **7/7**.

Not verified: that the banner actually clears on the phone — that needs the new APK installed on a real device (INST-24/25). What is proven is that the artefact now contains the policy Android requires, and that the setting cannot silently regress without a failing test.

---

8. Deliverables 13 & 14 — the authority backend and the real transport

This is the round this report's date was revised for: the app can now actually deliver events, and there is a service on the other end to receive and work them.

8.1 What the app sends

A new **authority client** (`app/src/main/java/com/touristsafety/app/authority/`) speaks the backend's HTTP contract: a hand-written JSON codec (`Json.kt` — used instead of `org.json`, which returns nulls on the plain JVM, so the whole client stays unit-testable), the wire codec, an `HttpURLConnection` layer, and a durable transport. The app's transport is now a **selector**: it uses the real authority transport when a base URL is configured and the honest `NoBackendTransport` otherwise, reading the address at flush time so **setting an address takes effect on the next flush with no restart**. The base URL and bearer token live in their own DataStore file (`authority_session`), separate from `AppSettings`.

Emergency contacts are **uploaded through the durable outbox** (`OutboxKind.CONTACTS`), not a direct `PUT`, so an offline edit still syncs when the network returns.

8.2 The delivery rules (why a mistake cannot lose an event)

The mapping from a server verdict onto the outbox is deliberately biased: only a *permanent* rejection discards an event.

Server outcome	Outbox result	Why
`DELIVERED`	**Delivered** (ack id)	stored by the authority
`DUPLICATE`	**Delivered**	idempotency — the server already has it
`REJECTED` (or 400/422)	**Rejected** with the reason	the same bytes can never be accepted
401 / 403	**Retry**	re-signing-in will deliver it
404 / 409 / 5xx / IO / malformed	**Retry**	could be a misconfiguration — never discard

8.3 The backend and the dashboard

`backend/` is one Flask app over one SQLite file, serving both the phone API and a static operator dashboard at `/`. It generates its own credentials on first start and writes them once to `backend/var/DEV_CREDENTIALS.txt`; **no password is written into the source**, and `test_security.py` asserts that. The `events` table is append-only with a `UNIQUE (tourist_id,`

`dedupe_key)` index — that index is what makes re-delivery harmless — and an SOS has its own table because it has a workflow (OPEN → ACKNOWLEDGED).

Run it with `python backend\run.py`; the dashboard is then at http://127.0.0.1:8080/. Point the phone at it in **Settings → Backend connection (advanced)**: `10.0.2.2:8080` from an emulator, or the printed LAN address from a physical phone. (A debug build can also bake a first-launch default in; this pass's APK carries `http://192.168.1.102:8080`.) Press **Test connection** and confirm it reports *Connected* — the screen only claims that after a real `GET /api/v1/health` answered. Until an address is set **and** a tourist is signed in, events stay queued and the app says so. Full detail: `backend/README.md`.

8.4 What was verified, and refused to be claimed

- **Backend rules:** `python -m pytest backend\tests -q` → **265 passed** (in-process Flask client, temp SQLite, frozen clock).
- **Backend acceptance, real subprocess:** `python backend\verify_integration.py` → **41/41 PASS** (self-registration and sign-in with the minted id + PIN, duplicate phone 409 / role escalation 400, delivery, idempotency, SOS acknowledgement, the future-risk prediction reaching `/authority/risk` while raising no SOS, restart persistence, dashboard fetch).
- **The production Kotlin client against a real server:** `AuthorityLiveBackendTest` **7/7 passed** — it **self-registered a tourist with no credential and received `TR-2026-00001`, signed in with that id and the PIN it chose, and confirmed the officer's register showed it as `SELF` / `REGISTERED` with no position before a real-GPS report turned the same row into `trackingStatus=OK`, `hasPosition=true`**; it also signed in as a real tourist, flushed a real SOS, got a server event id, saw `/me` report `delivered=1 latestSos=OPEN simulated=true`, confirmed **one** officer-queue row after a duplicate, delivered a **real-GPS position report** that landed on the authority map (`latestPosition: 18.5204,73.8567 simulated=false`), set a PIN and then signed in with it (a wrong PIN → HTTP 401), **signed in as an officer and read the register through the app's own console method while a tourist token was refused the same route with HTTP 403**, showed a dead backend keeps the event queued with a reason, and pushed an **on-device future-risk summary** through the production payload builder and queue: the authority stored it, labelled it `PREDICTION`, counted it as elevated risk (`riskAssessments=1`, `riskElevated=1`) while `sosTotal` **stayed at 1** — a forecast is not an incident.

NOT claimed: that it works *from a device*. The device's own `HttpURLConnection`, emulator loopback / Wi-Fi routing and the Compose Authority screen (including the new registration screen) were never exercised (INST-20, INST-21, INST-24). A simulated SOS is stored and badged as **simulated** everywhere; nothing is contacted automatically. Routine `POSITION` reporting now exists (item 15) behind an explicit, tourist-controlled interval/distance policy, but the on-device GPS that would drive it has not been run here.

---

9. Testing & verification

- Android JVM suite: **548 tests, 0 failures, 0 errors, 7 skipped** across **50 classes** (`:app:testDebugUnitTest`, re-run this round with `--rerun`). The seven skips are the opt-in live-backend tests, which run only when given `-Dtsm.e2e.baseUrl` (they last ran 7/7 in the previous round). The most recent round added **3 tests in one new class** — `NetworkSecurityConfigTest`, which pins the manifest + network-security-profile pair that lets a plain-HTTP LAN authority be reached at all (§7.13). The round before it added **30 tests across two new classes and four grown ones** for the two-role split and the dead-button fix — officer sign-in/console (`OfficerViewModelTest`, 10), the real on-disk session store (`AuthoritySessionStoreTest`, 8), per-field form errors (`RegistrationFormTest` 13 → 18), the officer register codec (`AuthorityApiTest` 24 → 28), the officer register over a real socket (`AuthorityHttpServerTest` 12 → 14) and a **seventh live end-to-end test** (`AuthorityLiveBackendTest` 6 → 7). The round before it added **31 tests across five classes** for self-service registration — the form rules (`RegistrationFormTest`, 13), the request/response codec and a real-socket registration (`AuthorityApiTest` 20 → 24, `AuthorityHttpServerTest` 10 → 12), the ViewModel state machine (`AuthorityViewModelTest`, 11) and a **sixth live end-to-end test** (`AuthorityLiveBackendTest` 5 → 6) — and an earlier round added **94** for the future-event model: the 22 features index by index (`FutureRiskFeaturesTest`, 13), **the shipped asset against scikit-learn's own probabilities on 16 real held-out windows** (`FutureRiskReferenceTest`, 6), inference/bands/staleness/insufficient data (`FutureRiskAnalyzerTest`, 18), the sharing policy (`FutureRiskReportPolicyTest`, 10), the durable outbox path (`FutureRiskReporterTest`, 12), the sensor integration (`FutureRiskMovementIntegrationTest`, 8), the wire contract (`FutureRiskPayloadTest`, 8) and what the card is allowed to say (`FutureRiskCardModelTest`, 18). Earlier rounds still covered routine position reporting, the PIN/position payloads, the authority client and its transport rules, a real-socket HTTP test, the JSON codec, the emergency-contact repository, the resume-recovery rule, the geofence walk-through, the on-screen diagnostics, the movement/fall pipeline, the approach-risk maths, the shipped-forest fidelity and the SOS delivery ordering rule.
- Authority backend: **265 tests** (`python -m pytest backend/tests -q`, executed this round — up from 220; the new `test_registration.py` contributes 45) plus a **41/41** real-subprocess acceptance run (`python backend/verify_integration.py`).
- Live client → real server: `AuthorityLiveBackendTest` **7/7 passed** on the JVM, driving the production Kotlin client against `python backend/run.py` (**self-registration → the server-minted Tourist ID → signing in with it → the officer's register showing it with no position → a real position arriving**, sign-in, outbox flush, server event id, `/me` totals, the officer's SOS queue, a delivered position report, PIN set + PIN sign-in, **an officer sign-in reading the register while a tourist token was refused it with 403**, a dead-backend retry, and an on-device future-risk summary that the authority stores but never turns into an incident).

- ML suite: **179 tests, 0 failures** (`python -m pytest ml/tests ml/future_risk/tests -q`) — 86 existing plus **93** new. The new `ml/future_risk/tests/test_future_risk.py` rebuilds the label geometry from the raw `full_data.mat`, re-derives the leakage checks, the calibration and the band rule, pins the provenance (including the corrupt recording and the licence position), re-applies the **model-selection rule from** `tuning.json`, and asserts the app asset and the reference cases are **byte-identical** to the pipeline's output.
- Export fidelity: `python ml/har_verify_export.py` → **ALL PASS** (max probability error 4.97e-07 activity, 4.74e-07 impact; 0 argmax mismatches). For the new model, `FutureRiskReferenceTest` is the equivalent check and it runs inside `:app:testDebugUnitTest`.
- Training reproducibility: re-running `ml/har_train.py` reproduced identical metrics and regenerated `har_activity_model.json` **byte-identical** to the shipped copy. Re-running `ml/future_risk/train.py` this round reproduced every metric to the digit (PR-AUC 0.764569, trial sensitivity 0.929577) and regenerated `future_risk_model.json` byte-identically.
- Instrumented suite: **compile-checked only** (`:app:compileDebugAndroidTestKotlin` successful) — **not executed** (no device/emulator; `adb devices` empty).
- Debug APK: **built successfully**, **20.33 MB** (21,317,914 bytes), SHA-256 `F0786379EC1917F9004E03F72E9317FB0C0F97B4B7D4BF9F312A911D3BE5D239`, built with `-PAUTHORITY_URL=http://192.168.1.102:8080` (present in the packaged dex) and declaring `android:networkSecurityConfig` → `res/xml/network_security_config.xml` (`cleartextTrafficPermitted="true"`), both confirmed by dumping the shipped manifest and resources.

The **149** enumerated acceptance cases (with objective, expected, actual, result, evidence) and the **25** NOT-EXECUTED cases are in `docs/test_report.md`. The migration path is now proven twice: the Robolectric `RoomPersistenceTest` **executes both migrations** on a real SQLite database on the JVM (so the non-destructive claim runs in every build), while the instrumented `DatabaseMigrationTest` (v1→v2, v2→v3 and v1→v3) is compile-checked only because it needs a device. The full live-integration transcript is in §7.1 of that report.

---

10. Files created / modified in this pass

New — Android main source

```
app/src/main/java/com/touristsafety/app/ui/map/MercatorProjection.kt
app/src/main/java/com/touristsafety/app/ui/map/TilePolicy.kt
app/src/main/java/com/touristsafety/app/ui/components/OfflineOsmMap.kt
app/src/main/java/com/touristsafety/app/ui/components/TileDiskCache.kt
app/src/main/java/com/touristsafety/app/ui/components/DeviceLocation.kt
app/src/main/java/com/touristsafety/app/geofencing/SampleAreas.kt
app/src/main/java/com/touristsafety/app/domain/model/OutboxEvent.kt
```

```
app/src/main/java/com/touristsafety/app/data/database/entity/OutboxEventEntity.kt
app/src/main/java/com/touristsafety/app/data/database/dao/OutboxEventDao.kt
app/src/main/java/com/touristsafety/app/data/mapper/OutboxEventMappers.kt
app/src/main/java/com/touristsafety/app/data/repository/OutboxRepository.kt
app/src/main/java/com/touristsafety/app/data/database/DatabaseMigrations.kt
app/src/main/java/com/touristsafety/app/outbox/OutboxPayloads.kt
app/src/main/java/com/touristsafety/app/outbox/SyncTransport.kt
app/src/main/java/com/touristsafety/app/outbox/OutboxSyncer.kt
app/src/main/java/com/touristsafety/app/watch/WatchUiState.kt
app/src/main/java/com/touristsafety/app/watch/WatchHaptics.kt
app/src/main/java/com/touristsafety/app/watch/AndroidWatchHaptics.kt
app/src/main/java/com/touristsafety/app/watch/WatchController.kt
app/src/main/java/com/touristsafety/app/ui/viewmodel/WatchViewModel.kt
app/src/main/java/com/touristsafety/app/ui/screens/WatchScreen.kt
app/src/main/java/com/touristsafety/app/diagnostics/CrashLog.kt                (crash recor
app/src/main/java/com/touristsafety/app/ui/components/CrashReportDialog.kt   (on-launch re
app/src/main/java/com/touristsafety/app/motion/MotionFeatures.kt             (24 features,
app/src/main/java/com/touristsafety/app/motion/DecisionForest.kt             (JSON forest
app/src/main/java/com/touristsafety/app/motion/MovementModels.kt             (asset loader
app/src/main/java/com/touristsafety/app/motion/MovementAnalyzer.kt           (windows + th
app/src/main/java/com/touristsafety/app/motion/MotionSource.kt               (sensor abstr
app/src/main/java/com/touristsafety/app/motion/SensorMotionSource.kt         (acceleromete
app/src/main/java/com/touristsafety/app/motion/MovementController.kt         (watch lifecy
app/src/main/java/com/touristsafety/app/geofencing/ApproachRisk.kt           (published we
app/src/main/java/com/touristsafety/app/geofencing/ApproachBriefing.kt       (which zone,
app/src/main/java/com/touristsafety/app/ui/components/MovementAndApproachCards.kt (the two
app/src/main/java/com/touristsafety/app/authority/Json.kt                     (hand-rolled
app/src/main/java/com/touristsafety/app/authority/AuthorityContract.kt
app/src/main/java/com/touristsafety/app/authority/AuthorityApi.kt             (the wire co
app/src/main/java/com/touristsafety/app/authority/AuthorityHttp.kt            (HttpURLConn
app/src/main/java/com/touristsafety/app/authority/AuthorityClient.kt
app/src/main/java/com/touristsafety/app/authority/AuthoritySessionStore.kt    (its own Dat
app/src/main/java/com/touristsafety/app/authority/AuthoritySyncTransport.kt   (+ Selecting
app/src/main/java/com/touristsafety/app/domain/model/EmergencyContact.kt
app/src/main/java/com/touristsafety/app/data/database/entity/EmergencyContactEntity.kt
app/src/main/java/com/touristsafety/app/data/database/dao/EmergencyContactDao.kt
app/src/main/java/com/touristsafety/app/data/mapper/EmergencyContactMappers.kt
app/src/main/java/com/touristsafety/app/data/repository/EmergencyContactRepository.kt
app/src/main/java/com/touristsafety/app/ui/viewmodel/AuthorityViewModel.kt
app/src/main/java/com/touristsafety/app/ui/viewmodel/EmergencyContactsViewModel.kt
app/src/main/java/com/touristsafety/app/ui/screens/AuthorityScreen.kt
app/src/main/java/com/touristsafety/app/ui/screens/EmergencyContactsScreen.kt

# --- future-event risk model (on-device inference, this round) ---
app/src/main/java/com/touristsafety/app/motion/FutureRiskFeatures.kt      (22 features, th
app/src/main/java/com/touristsafety/app/motion/FutureRiskModel.kt         (asset loader +
```

```
app/src/main/java/com/touristsafety/app/motion/FutureRiskAnalyzer.kt      (bands, stalenes
app/src/main/java/com/touristsafety/app/motion/FutureRiskReporter.kt      (when a summary
app/src/main/java/com/touristsafety/app/ui/components/FutureRiskCardModel.kt (what the car
app/src/main/java/com/touristsafety/app/ui/components/FutureRiskCard.kt   (thin renderer o

# --- LAN cleartext policy (this round) ---
app/src/main/res/xml/network_security_config.xml                          (cleartextTraffi
```

Modified — Android main source

```
app/src/main/java/com/touristsafety/app/ui/components/SafetyMap.kt            (trail, Foll
app/src/main/java/com/touristsafety/app/ui/components/OfflineOsmMap.kt        (blue marker
app/src/main/java/com/touristsafety/app/ui/components/DeviceLocation.kt       (last-known
app/src/main/java/com/touristsafety/app/ui/map/TilePolicy.kt                  (providersTo
app/src/main/java/com/touristsafety/app/ui/theme/Color.kt                     (UserMarkerA
app/src/main/java/com/touristsafety/app/MainActivity.kt                       (crash dialo
app/src/main/java/com/touristsafety/app/TouristSafetyApp.kt                   (CrashLog.in
app/src/main/java/com/touristsafety/app/ui/screens/TrackingScreen.kt          (trail, foll
app/src/main/java/com/touristsafety/app/ui/screens/GeoFenceEditorScreen.kt    (opens on th
app/src/main/java/com/touristsafety/app/ui/viewmodel/GeoFenceEditorViewModel.kt (addVertex
app/src/main/java/com/touristsafety/app/ui/screens/ManageZonesScreen.kt       (sample-area
app/src/main/java/com/touristsafety/app/ui/viewmodel/ManageZonesViewModel.kt  (addSampleAr
app/src/main/java/com/touristsafety/app/tracking/TrackingController.kt
app/src/main/java/com/touristsafety/app/tracking/LocationTrackingService.kt
app/src/main/java/com/touristsafety/app/ui/viewmodel/HomeViewModel.kt
app/src/main/java/com/touristsafety/app/ui/screens/HomeScreen.kt
app/src/main/java/com/touristsafety/app/ui/viewmodel/TrackingViewModel.kt     (movement, a
app/src/main/java/com/touristsafety/app/watch/WatchController.kt              (re-read aft
app/src/main/java/com/touristsafety/app/data/database/AppDatabase.kt          (VERSION 1 →
app/src/main/java/com/touristsafety/app/AppContainer.kt                       (motionSourc
app/src/main/java/com/touristsafety/app/ui/viewmodel/ViewModelFactory.kt
app/src/main/java/com/touristsafety/app/navigation/AppNavigation.kt           (WATCH, AUTH
app/src/main/AndroidManifest.xml                                              (VIBRATE)
# --- this round: the future-event model ---
app/src/main/java/com/touristsafety/app/motion/DecisionForest.kt              (optional ex
app/src/main/java/com/touristsafety/app/motion/MovementController.kt          (futureRiskM
app/src/main/java/com/touristsafety/app/AppContainer.kt                       (loads futur
app/src/main/java/com/touristsafety/app/ui/viewmodel/ViewModelFactory.kt      (passes the
app/src/main/java/com/touristsafety/app/ui/viewmodel/TrackingViewModel.kt     (exposes fut
app/src/main/java/com/touristsafety/app/ui/screens/TrackingScreen.kt          (renders Fut
app/src/main/java/com/touristsafety/app/domain/model/OutboxEvent.kt           (+ OutboxKin
app/src/main/java/com/touristsafety/app/outbox/OutboxPayloads.kt              (+ FutureRis
```

```
# --- this round: the LAN cleartext policy ---
app/src/main/AndroidManifest.xml                                          (+ android:n
```

Assets (shipped models, new)

```
app/src/main/assets/har_activity_model.json       (262.4 KB — 32-tree forest, 24 features
app/src/main/assets/har_fall_model.json           ( 58.3 KB — 40-tree impact detector)
app/src/main/assets/har_activity_model_spec.json  ( 3.5 KB — model card data)
app/src/main/assets/future_risk_model.json        (  1.88 MB — 120-tree future-event fore
                                                   Platt calibration, band thresholds, fe
```

Tests

```
NEW  app/src/test/java/com/touristsafety/app/ui/map/MercatorProjectionTest.kt
NEW  app/src/test/java/com/touristsafety/app/ui/map/TilePolicyTest.kt
NEW  app/src/test/java/com/touristsafety/app/ui/components/TileDiskCacheTest.kt
NEW  app/src/test/java/com/touristsafety/app/geofencing/SampleAreasTest.kt
NEW  app/src/test/java/com/touristsafety/app/outbox/OutboxQueueTest.kt
NEW  app/src/test/java/com/touristsafety/app/outbox/OutboxPayloadsTest.kt
NEW  app/src/test/java/com/touristsafety/app/outbox/FakeSyncTransport.kt
NEW  app/src/test/java/com/touristsafety/app/watch/WatchControllerTest.kt
NEW  app/src/test/java/com/touristsafety/app/watch/SosDeliveryStateTest.kt                (
NEW  app/src/test/java/com/touristsafety/app/motion/ForestReferenceTest.kt               (s
NEW  app/src/test/java/com/touristsafety/app/motion/MovementAnalyzerTest.kt
NEW  app/src/test/java/com/touristsafety/app/motion/MovementControllerTest.kt
NEW  app/src/test/java/com/touristsafety/app/motion/MotionTestFixtures.kt
NEW  app/src/test/java/com/touristsafety/app/geofencing/ApproachRiskTest.kt
NEW  app/src/test/java/com/touristsafety/app/geofencing/ApproachBriefingTest.kt
NEW  app/src/test/resources/har_activity_reference_cases.json                            (r
NEW  app/src/test/resources/har_fall_reference_cases.json
NEW  app/src/test/java/com/touristsafety/app/authority/JsonTest.kt
NEW  app/src/test/java/com/touristsafety/app/authority/AuthorityApiTest.kt
NEW  app/src/test/java/com/touristsafety/app/authority/AuthoritySyncTransportTest.kt
NEW  app/src/test/java/com/touristsafety/app/authority/AuthorityHttpServerTest.kt
NEW  app/src/test/java/com/touristsafety/app/authority/AuthorityLiveBackendTest.kt
NEW  app/src/test/java/com/touristsafety/app/data/repository/EmergencyContactRepositoryTes
MOD  app/src/test/java/com/touristsafety/app/data/database/RoomPersistenceTest.kt
MOD  app/src/androidTest/java/com/touristsafety/app/data/database/DatabaseMigrationTest.kt
MOD  app/src/test/java/com/touristsafety/app/outbox/OutboxPayloadsTest.kt
MOD  app/src/test/java/com/touristsafety/app/ui/viewmodel/GeoFenceEditorViewModelTest.kt
MOD  app/src/test/java/com/touristsafety/app/testing/TestSupport.kt                       (
MOD  app/src/test/java/com/touristsafety/app/ui/map/TilePolicyTest.kt                     (
```

```
NEW  app/src/main/java/com/touristsafety/app/tracking/PositionReporter.kt
NEW  app/src/test/java/com/touristsafety/app/tracking/PositionReporterTest.kt
NEW  app/src/test/java/com/touristsafety/app/tracking/PositionReportPolicyTest.kt
MOD  app/src/main/java/com/touristsafety/app/outbox/OutboxPayloads.kt
MOD  app/src/main/java/com/touristsafety/app/authority/AuthorityClient.kt / AuthorityApi.k
MOD  app/src/main/java/com/touristsafety/app/ui/screens/AuthorityScreen.kt
MOD  app/src/main/java/com/touristsafety/app/ui/screens/HomeScreen.kt
MOD  app/src/main/java/com/touristsafety/app/ui/screens/{Settings,Simulation,GeoFenceEdito
# --- future-event model tests (93 new across 8 classes, plus 1 live E2E case, this round)
MOD  app/src/test/java/com/touristsafety/app/authority/AuthorityLiveBackendTest.kt
NEW  app/src/test/java/com/touristsafety/app/motion/FutureRiskFeaturesTest.kt            (
NEW  app/src/test/java/com/touristsafety/app/motion/FutureRiskReferenceTest.kt           (
NEW  app/src/test/java/com/touristsafety/app/motion/FutureRiskAnalyzerTest.kt            (
NEW  app/src/test/java/com/touristsafety/app/motion/FutureRiskReportPolicyTest.kt        (
NEW  app/src/test/java/com/touristsafety/app/motion/FutureRiskReporterTest.kt            (
NEW  app/src/test/java/com/touristsafety/app/motion/FutureRiskMovementIntegrationTest.kt (
NEW  app/src/test/java/com/touristsafety/app/motion/FutureRiskTestArtefact.kt            (
NEW  app/src/test/java/com/touristsafety/app/outbox/FutureRiskPayloadTest.kt             (
NEW  app/src/test/java/com/touristsafety/app/ui/components/FutureRiskCardModelTest.kt    (
NEW  app/src/test/resources/future_risk_reference_cases.json                             (
# --- LAN cleartext policy (this round) ---
NEW  app/src/test/java/com/touristsafety/app/NetworkSecurityConfigTest.kt
```

ML module (new)

```
ml/common.py  ml/preprocess.py  ml/eda.py  ml/train.py  ml/evaluate.py  ml/predict.py
ml/requirements.txt  ml/run_all.ps1
ml/data/README.md  ml/data/make_synthetic_fixture.py  ml/data/synthetic_sample/**
ml/tests/conftest.py  ml/tests/test_pipeline.py
ml/outputs/**  (clean/, eda/, train/, eval/, predict/)
ml/models/isolation_forest.joblib  ml/models/model_meta.json  ml/models/split.json

# --- movement / impact pipeline (UCI HAR, supervised) ---
ml/har_preprocess.py            (extract, integrity-check, subject-disjoint split)
ml/har_train.py                 (24-feature activity forest, per-class report, figures, ex
ml/har_fall.py                  (synthetic-impact detector + shuffled-label control)
ml/har_verify_export.py         (exported JSON vs scikit-learn: ALL PASS)
ml/tests/test_har_models.py     (21 contract tests for the shipped artefacts)
ml/outputs/har/clean/**         (metadata, class balance, subject split)
ml/outputs/har/train/**         (reports, metrics CSV, 5 figures, model JSON, spec)
ml/outputs/har/fall/**          (report, ROC/confusion/signal figures, model JSON)

# --- remaining datasets + history (this round) ---
ml/osm_eda.py                   (real OpenStreetMap EDA via the Overpass API; NOT_RUN on t
```

```
ml/weather_eda.py               (real weather EDA — Open-Meteo by default, OpenWeatherMap
ml/har_eda.py                   (UCI HAR figure set)
ml/history.py                   (historical trajectory + temporal + zone-crossing trends)
ml/transport_mode.py            (supervised transport-mode model — NOT a risk model)
ml/tests/test_osm_eda.py (12)  ml/tests/test_weather_eda.py (11)
ml/tests/test_history.py (13)  ml/tests/test_transport_mode.py (8)
ml/outputs/osm/**  ml/outputs/weather/**  ml/outputs/har/eda/**  ml/outputs/history/**  ml

# --- supervised FUTURE-event model (this round) ---
ml/future_risk/__init__.py
ml/future_risk/config.py        (the selected hyper-parameters, the size budget, the paths
ml/future_risk/features.py      (the 22 features + their human labels and formulas — the r
ml/future_risk/dataset.py       (raw-trial reader, sanitiser, impact finder, window cutter
ml/future_risk/train.py         (subject split, training, calibration, bands, metrics, fig
ml/future_risk/tune.py          (65-config validation search under the deployment size bud
ml/future_risk/tests/conftest.py        ml/future_risk/tests/test_future_risk.py (93)
ml/data/unimib/README section   (ml/data/README.md §6: the dataset, its licence position,
ml/outputs/future_risk/**       (windows.npz, dataset_provenance.json, metrics.json, tunin
                                 calibration_reliability.csv, test_predictions.csv, 6 figu
                                 future_risk_model.json, future_risk_reference_cases.json)
ml/run_all.ps1                  (MOD: + the `-FutureRisk` stage and the new test directory
ml/data/README.md               (MOD: §6 documents UniMiB SHAR, its licence position and t
```

Authority backend (new)

```
backend/README.md                 (how to run it, the API, the sync contract, the payload
backend/run.py                    (entry point: --host --port --database)
backend/pytest.ini
backend/tsm/__init__.py           (config: token TTL, seed accounts, demo ids, version)
backend/tsm/app.py                (Flask app factory, error envelope, static dashboard)
backend/tsm/api.py                (routes, auth, sync results, batch/page ceilings)
backend/tsm/payloads.py           (per-kind validators the phone's bytes must satisfy)
backend/tsm/db.py                 (idempotent SQLite schema, append-only events)
backend/tsm/security.py           (scrypt salt+digest; nothing recoverable stored)
backend/tsm/seed.py               (first-start credentials, written once)
backend/tsm/clock.py              (injectable clock, so expiry is testable)
backend/tsm/static/index.html · app.js · style.css   (the operator dashboard)
backend/tests/conftest.py         (FrozenClock, app_factory, short_ttl)
backend/tests/test_payloads.py (42) · test_authority.py (33) · test_sync.py (29)
backend/tests/test_auth.py (26) · test_pin.py (18) · test_app.py (16)
backend/tests/test_contacts.py (15) · test_security.py (13)
backend/tests/test_risk.py (28)   (the RISK payload: ingest, idempotency, read-back, 12 re
backend/tests/test_registration.py (45)   (self-registration: DB-minted unique ids, hashed
                                           validation, duplicate phone 409, no privilege es
```

```
                                        listing, registration-vs-tracking-vs-alert separ
backend/verify_integration.py     (41-check acceptance run against a real subprocess serve
backend/var/                      (generated: authority.db + DEV_CREDENTIALS.txt — not com
```

Authority backend, added this round (predictions, not incidents)

```
MOD  backend/tsm/payloads.py      (+ the RISK kind: band ceiling, nullable probability, ho
                                   5-factor ceiling, data-quality block — 12 rejection pat
MOD  backend/tsm/db.py            (+ the risk_assessments table and its two indexes)
MOD  backend/tsm/api.py           (+ prediction ingest, /me.latestRisk, /authority/risk, p
                                   overview riskAssessments/riskElevated; `state` delibera
MOD  backend/tsm/static/index.html (+ the "Future-risk predictions" panel, a Risk (model)
MOD  backend/tsm/static/app.js    (+ riskBandPill/renderRisk, risk tiles, risk cell, non-i
MOD  backend/tsm/static/style.css (+ .tile.warn, td.factors)
NEW  backend/tests/test_risk.py   (28)
MOD  backend/tests/test_payloads.py (the kinds test now names all five the phone can send)
MOD  backend/verify_integration.py (the accepted batch now carries a RISK summary too, so
                                   run proves a prediction reaches /authority/risk and rai
                                   26 → 33 checks)
```

Self-service tourist registration, added this round

```
NEW  backend/tsm/registration.py   (pure validation + the DB-backed Tourist-ID minting seq
MOD  backend/tsm/db.py             (+ the tourist_id_sequence table; + the nullable touris
MOD  backend/tsm/api.py            (+ POST /auth/register; registered_tourist_json; the en
                                   /authority/tourists register; the RISK insert moved ins
MOD  backend/tsm/static/index.html (+ the panel renamed to "Registered Tourists", new colu
                                   registration date + source, registration status, tracki
MOD  backend/tsm/static/app.js     (+ renderTourists now reads /authority/tourists; regist
                                   the tile relabelled; refresh() fetches the register)
NEW  backend/tests/test_registration.py (45)
MOD  backend/tests/test_app.py     (the schema list now names tourist_id_sequence)
MOD  backend/verify_integration.py (33 → 41 checks: self-register, the minted id, no token
                                   id + PIN, duplicate phone 409, role escalation 400, the
                                   it with no position, and it survives a restart)
NEW  app/src/main/java/com/touristsafety/app/authority/RegistrationForm.kt
                                   (pure client validation + RegistrationDraft/Registratio
MOD  app/src/main/java/com/touristsafety/app/authority/AuthorityContract.kt (+ REGISTER, A
MOD  app/src/main/java/com/touristsafety/app/authority/AuthorityApi.kt      (+ registerBod
MOD  app/src/main/java/com/touristsafety/app/authority/AuthorityClient.kt   (+ register(),
MOD  app/src/main/java/com/touristsafety/app/ui/viewmodel/AuthorityViewModel.kt (+ registr
MOD  app/src/main/java/com/touristsafety/app/ui/screens/AuthorityScreen.kt  (+ the Sign in
```

```
                                  the form with add-and-remove contacts, and the success
NEW  app/src/test/java/com/touristsafety/app/authority/RegistrationFormTest.kt (13)
NEW  app/src/test/java/com/touristsafety/app/ui/viewmodel/AuthorityViewModelTest.kt (11)
MOD  app/src/test/java/com/touristsafety/app/authority/AuthorityApiTest.kt         (20 → 2
MOD  app/src/test/java/com/touristsafety/app/authority/AuthorityHttpServerTest.kt  (10 → 1
MOD  app/src/test/java/com/touristsafety/app/authority/AuthorityLiveBackendTest.kt (5 → 6:
```

Two-role sign-in and the dead-button fix, added this pass

```
NEW  app/src/main/java/com/touristsafety/app/ui/screens/WelcomeScreen.kt     (the role cho
NEW  app/src/main/java/com/touristsafety/app/ui/screens/TouristAuthScreen.kt (tourist sign
NEW  app/src/main/java/com/touristsafety/app/ui/screens/OfficerScreen.kt     (officer sign
NEW  app/src/main/java/com/touristsafety/app/ui/viewmodel/OfficerViewModel.kt
MOD  app/build.gradle.kts            (+ BuildConfig.DEFAULT_AUTHORITY_URL from -PAUTHORITY
MOD  app/src/main/java/com/touristsafety/app/AppContainer.kt (start() seeds the default ad
MOD  app/src/main/java/com/touristsafety/app/authority/AuthorityContract.kt  (+ AUTHORITY_
                                  OfficerSession, AuthorityTouristRow; AuthorityTourist
MOD  app/src/main/java/com/touristsafety/app/authority/AuthoritySessionStore.kt (+ current
                                  clearOfficerLogin/seedBaseUrlIfUnset; separate office
                                  createAuthoritySessionStore(context) as the productio
MOD  app/src/main/java/com/touristsafety/app/authority/AuthorityApi.kt       (+ parseTouri
MOD  app/src/main/java/com/touristsafety/app/authority/AuthorityClient.kt    (+ authorityT
MOD  app/src/main/java/com/touristsafety/app/authority/RegistrationForm.kt   (+ Registrati
MOD  app/src/main/java/com/touristsafety/app/ui/viewmodel/AuthorityViewModel.kt (+ connect
MOD  app/src/main/java/com/touristsafety/app/ui/viewmodel/HomeViewModel.kt   (+ authorityC
MOD  app/src/main/java/com/touristsafety/app/ui/viewmodel/ViewModelFactory.kt (+ OfficerVi
MOD  app/src/main/java/com/touristsafety/app/ui/screens/AuthorityScreen.kt   (now the adva
                                  and registration removed — this is the file that held
MOD  app/src/main/java/com/touristsafety/app/ui/screens/HomeScreen.kt        (the upload c
                                  the raw backend address left the tourist path)
MOD  app/src/main/java/com/touristsafety/app/navigation/AppNavigation.kt     (WELCOME / TO
                                  session-aware start gate, no chrome on onboarding; cl
MOD  app/src/main/java/com/touristsafety/app/ui/screens/SettingsScreen.kt    ("Connection
NEW  app/src/test/java/com/touristsafety/app/ui/viewmodel/OfficerViewModelTest.kt (10)
NEW  app/src/test/java/com/touristsafety/app/authority/AuthoritySessionStoreTest.kt (8, re
MOD  app/src/test/java/com/touristsafety/app/authority/RegistrationFormTest.kt      (13 →
MOD  app/src/test/java/com/touristsafety/app/authority/AuthorityApiTest.kt          (24 →
MOD  app/src/test/java/com/touristsafety/app/authority/AuthorityHttpServerTest.kt   (12 →
MOD  app/src/test/java/com/touristsafety/app/authority/AuthorityLiveBackendTest.kt  (6 → 7
MOD  app/src/test/java/com/touristsafety/app/testing/Fakes.kt (InMemoryAuthoritySessionSto
MOD  README.md · docs/test_report.md · docs/final_project_report.md · docs/architecture.md
                                  (the flow, the route map, the measured counts and the
                                  regenerated from the .md sources)
```

No backend file changed this pass: `GET /api/v1/authority/tourists` already required `require_auth("authority")`, so the officer console reads the existing route and the role boundary is unchanged server-side.

Docs & scripts

```
docs/architecture.md            (existing, updated)
docs/dataset_and_eda.md         (new)
docs/model_card.md              (new — GeoLife trajectory anomaly)
docs/model_card_movement.md     (new — activity + impact models, metrics, confusion matric
docs/google_maps_setup.md       (new — how to supply your own Google Maps key)
docs/test_report.md             (new)
docs/final_project_report.md    (new — this file)
docs/progress_log.md            (updated)
README.md                       (updated)
build-and-install.bat           (new — one-click build + install)
docs/future_risk_prediction.md  (new — model card + dataset provenance for the future-even
```

---

11. Exact commands

```
# ---- Android ----
cd D:\TouristSafetyMonitoring-src
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 ":app:testDebugUnitTest"
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 ":app:assembleDebug"
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 ":app:compileDebugAndroidT
# with a device/emulator:
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 ":app:connectedDebugAndroi

# ship a first-launch authority address (optional; never overwrites a chosen one)
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 ":app:assembleDebug" "-PAU

# optional Google Maps key (not committed; passed for one build only)
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 ":app:assembleDebug" "-PMA

# ---- authority backend + dashboard ----
python backend\run.py            # dashboard http://127.0.0.1:8080/  · health /api/v1/heal
python backend\run.py --host 0.0.0.0   # reachable from a phone on the same network
python -m pytest backend\tests -q        # 265 pass
python backend\verify_integration.py     # 41/41 PASS against a real subprocess server

# ---- optional: drive the production Kotlin client at a running server ----
```

```
python backend\run.py --port 8080        # terminal 1
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 ":app:testDebugUnitTest" `
  "--tests" "com.touristsafety.app.authority.AuthorityLiveBackendTest" `
  "-Dtsm.e2e.baseUrl=http://127.0.0.1:8080" `
  "-Dtsm.e2e.credentials=D:\TouristSafetyMonitoring-src\backend\var\DEV_CREDENTIALS.txt"

# ---- install ----
D:\AndroidTools\Sdk\platform-tools\adb.exe install -r app\build\outputs\apk\debug\app-debu

# ---- ML ----
cd ml
pip install -r requirements.txt
python -m pytest tests -q
python -m pytest ../ml/tests ../ml/future_risk/tests -q
powershell -NoProfile -ExecutionPolicy Bypass -File .\run_all.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File .\run_all.ps1 -Network -Transport
powershell -NoProfile -ExecutionPolicy Bypass -File .\run_all.ps1 -FutureRisk
python osm_eda.py
python weather_eda.py
python history.py
python transport_mode.py
python future_risk/train.py
python future_risk/tune.py
python har_train.py
python har_fall.py
python har_verify_export.py
python predict.py --plt "data\geolife\Geolife Trajectories 1.3\Data\000\Trajectory\2008102
```

The APK is at `app\build\outputs\apk\debug\app-debug.apk` (20.33 MB / 21,317,914 bytes, SHA-256 `F0786379EC1917F9004E03F72E9317FB0C0F97B4B7D4BF9F312A911D3BE5D239`; copies placed in the project root, on the Desktop and in `Downloads` as `app-debug.apk`). It is built with `-PAUTHORITY_URL=http://192.168.1.102:8080`, so a first launch on this machine's Wi-Fi is already pointed at the server; the address can still be changed in *Settings → Backend connection (advanced)* and a build-time default never overwrites a chosen one. Install it **over** the existing app (`adb install -r`, or tap the APK without uninstalling): the signing key and package are unchanged, so the saved address and sessions survive — and it is the only build that contains the cleartext fix of §7.13.

---

12. Limitations & blockers (stated, not hidden)

1. **No device/emulator here.** Map tile rendering, the crash fix, the new movement/approach cards, live sensor inference, the backend/contacts screens, the new **Welcome**, **tourist sign-in / create-account** and **officer console** screens, and all `androidTest` sources (including the

migration test) are **NOT EXECUTED** — compile- and JVM-checked only. `adb devices` is empty. Recorded as INST-16…INST-25 in `docs/test_report.md`.

2. **The backend exists, but the device's own path to it is unproven.** `backend/` is a working Flask + SQLite service with an operator dashboard, and the production Kotlin client was driven against it on the JVM (41/41 acceptance + 7/7 live). What has **not** been done here is the same flow *from a device* — its `HttpURLConnection`, emulator loopback `10.0.2.2` / Wi-Fi routing, and the Compose sign-in, create-account, role-chooser and officer-console screens (INST-20, INST-21, INST-24, INST-25). Routine `POSITION` reporting now exists (behind an explicit interval/distance policy) but its on-device GPS has not been run here. **No SOS is ever sent automatically.**
3. **The movement impact detector uses no real falls.** UCI HAR contains none, so that detector's positives are **synthesised impacts injected into real recordings**; its 0.9999 ROC-AUC measures detection of that injected signature, **not** clinical fall detection. Controls are in `docs/model_card_movement.md`. **This is no longer true of the future-risk model**: §7.10's predictor is trained on **real recorded falls**, and its 0.764569 PR-AUC / 0.929577 trial sensitivity are real held-out numbers — but they are still *fall* numbers from a lab, not tourist-incident numbers, and it is **not a deployment-ready alarm** (5.09 false alarms per minute of ordinary activity; see item 10).
4. **GeoLife has no real anomaly labels.** Accuracy/F1 against real anomalies is impossible; only an injection-based ROC-AUC (0.8922) is reported, and the trajectory model is unsupervised by design. The output is a *movement anomaly score*, never danger.
5. **The approach-risk score is a weighted checklist, not a fitted model.** Published weights, unit-tested, and labelled as such in the UI. It has no labelled outcome data to learn from.
6. **Geographic bias** in the GeoLife data (dominated by a few Beijing users), and **placement bias** in UCI HAR (30 volunteers, waist-mounted phone) mean "normal" is region-, person- and posture-specific.
7. **The key-less map is not Google Maps.** Real Google tiles require the user's own API key (`docs/google_maps_setup.md`); without one, the app uses key-free basemaps (OpenStreetMap by default, with Esri and OpenTopoMap as fallbacks). CARTO Voyager was tried and removed because it now serves a watermark instead of a map, so it is not offered at all.
8. **PDF/HTML are best-effort exports.** Every document has a sibling `.html` and `.pdf` in `docs\` (`README.html` / `README.pdf` live in the repository root) (Markdown → HTML via `python-markdown` → PDF via headless Microsoft Edge `--print-to-pdf`). **All companions were regenerated on 2026-10-18**, including the new `docs/future_risk_prediction.html` / `.pdf`, so they match the Markdown of this round. Prefer the Markdown as the source of truth. The PDFs embed the model-card figures from `ml/outputs/har/**`, so the `.md` and the figures must be copied together.
9. This is a **monitoring aid**, not an emergency service: it does not contact anyone automatically.
10. **The future-risk model is not a deployable fall alarm.** Measured on held-out subjects it raises **5.09 false alarms per minute** of ordinary activity at the app's cadence (16.30/min at the

raw window stride), concentrated in *jumping* (0.161), *sitting down* (0.156) and *lying down* (0.127); its mid-range probabilities (0.2–0.4) are over-confident; it was trained on 30 healthy adults in a lab carrying or pocketing a phone, never on a tourist outdoors; it forecasts **one second** ahead and nothing longer; and it **cannot distinguish a dropped phone from a falling person** (no phone-drop activity exists in the dataset, so that failure mode is untested). It is **not the best model in the repository** — `hist_gradient_boosting` reaches PR-AUC 0.8150 but cannot be walked on-device. A HIGH band is an **informational UI state** and a queued summary: it raises **no** alarm, changes **no** operational state, and **never** sends an SOS. Full detail in `docs/future_risk_prediction.md`.

11. **The future-risk card has never been rendered on a screen.** No device or emulator exists here, so the Compose card, the real sensor→forest→card chain and the dashboard's new predictions panel are **NOT EXECUTED** (INST-22). The wording is instead pinned by `FutureRiskCardModelTest` (18 tests) and the inference by `FutureRiskReferenceTest`, which replays the shipped asset against scikit-learn's own probabilities on 16 real held-out windows.

---

13. Recommendations / next steps

1. Install the APK on a physical phone and run INST-01…INST-21 (`docs/test_report.md`), especially INST-16 (the reported crash), INST-17 (live sensor inference) and INST-20/INST-21 (the Authority screen and the device's own path to the backend).
2. Retrain the **movement impact detector** on real falls too (SisFall / MobiFall), or fold §7.10's future-risk model into the reaction path once its false-alarm rate is acceptable — that is the single change that would turn the remaining impact-detector numbers into a fall-detection claim. The future-risk model already uses real falls (UniMiB SHAR), but it forecasts one second ahead rather than detecting the impact.
3. The authority endpoint now exists (`backend/`); the remaining work there is deployment, not a feature: put it behind TLS and a real reverse proxy, move off the Flask dev server, back up the SQLite file, and add online token revocation/rotation. Then decide a **position-report frequency policy** so the dashboard's "latest position" tracks the tourist without draining the battery.
4. Add a proper AVD/CI device so the instrumented suite (migration, persistence, UI) runs automatically.
5. Re-train the trajectory anomaly model on data representative of the deployment region before any real use, and keep it strictly advisory alongside geo-fencing.
6. Supply a Google Maps API key if the tool under test requires the real Google basemap (free tier, ~2 minutes — `docs/google_maps_setup.md`).